Javascript Programs

1) Rotten Oranges problem

The "Rotting Oranges" problem is a typical problem involving breadth-first search (BFS). The goal is to determine how long it takes for all fresh oranges to rot, given that rotting oranges can rot adjacent fresh oranges in one unit of time.

Problem Statement:

Given a 2D grid of integers where:

- `0` represents an empty cell,
- `1` represents a fresh orange,
- `2` represents a rotten orange,

You need to find the minimum number of minutes it takes for all fresh oranges to rot. If it is impossible to rot all the oranges, return `-1`.

Approach:

1. Use BFS to simulate the rotting process.
2. Start by enqueuing all initially rotten oranges.
3. At each minute, spread the rot to adjacent cells (up, down, left, right) if they contain fresh oranges.
4. Count how many minutes it takes for all oranges to rot.
5. If there are any remaining fresh oranges after the BFS, return `-1`.

Code Implementation in JavaScript:

Copy code

```
function orangesRotting(grid) {
  let rows = grid.length;
  let cols = grid[0].length;
  let queue = [];
  let freshOranges = 0;
  let minutes = 0;

  // Directions for moving in 4 directions (up, down, left, right)
  const directions = [
```

```
    [0, 1], // right
    [1, 0], // down
    [0, -1], // left
    [-1, 0]  // up
  ];

  // Step 1: Add all initially rotten oranges to the queue and count fres
  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      if (grid[r][c] === 2) {
        queue.push([r, c]); // Enqueue rotten oranges
      } else if (grid[r][c] === 1) {
        freshOranges++; // Count fresh oranges
      }
    }
  }

  // If there are no fresh oranges, return 0 minutes as there's nothing t
  if (freshOranges === 0) return 0;

  // Step 2: Perform BFS to spread the rot to adjacent fresh oranges
  while (queue.length > 0) {
    let nextQueue = [];

    while (queue.length > 0) {
      let [r, c] = queue.shift();

      // Try all 4 possible directions
      for (let [dr, dc] of directions) {
        let newR = r + dr;
        let newC = c + dc;

        // Check if the new position is within bounds and is a fresh oran
        if (newR >= 0 && newR < rows && newC >= 0 && newC < cols && grid[
          grid[newR][newC] = 2; // Rot the fresh orange
          freshOranges--; // Reduce the fresh orange count
          nextQueue.push([newR, newC]); // Add it to the queue for the ne
        }
      }
    }

    // If any oranges were rotted in this round, increment the time.
    if (nextQueue.length > 0) {
      minutes++;
    }

    // Update the queue to the next set of oranges to be processed.
    queue = nextQueue;
  }
```

```
  // If there are still fresh oranges, return -1, otherwise return the mi
  return freshOranges === 0 ? minutes : -1;
}
```

Explanation:

1. **Initial Setup**:

- Traverse the grid to find all rotten oranges (`2`s) and add their coordinates to the queue.
- Count the number of fresh oranges (`1`s).

2. **Breadth-First Search (BFS)**:

- At each minute, we process the rotten oranges in the queue, spreading the rot to all adjacent fresh oranges.
- The newly rotten oranges are added to the queue for the next round.
- The time increments after each complete round of rotting.

3. **Termination**:

- If there are no more fresh oranges by the end, return the total number of minutes.
- If there are still fresh oranges that cannot rot (e.g., isolated by empty cells), return `-1`.

Example:

Copy code

```
let grid = [
  [2, 1, 1],
  [1, 1, 0],
  [0, 1, 1]
];

console.log(orangesRotting(grid)); // Output: 4
```

Time and Space Complexity:

- **Time Complexity**: O(n * m), where `n` is the number of rows and `m` is the number of columns, because we traverse each cell at most once.

- **Space Complexity**: O(n * m) for the queue used in BFS.

2) Max Profit if we sell on particular day maxProfit([7,1,5,3,6,4])

```
var maxProfit = function(prices) {
    let maxprofit = 0;
    for (let i = 1; i < prices.length; i++) {
        if (prices[0] > prices[i]) prices[0] = prices[i];
        maxprofit = Math.max(maxprofit, prices[i] - prices[0]);
    }
    return maxprofit;
};

maxProfit([7,1,5,3,6,4])
```

3) Two Sum Problem

```
function twoSum(nums, target) {
  let numToIndex = {};

  for (let i = 0; i < nums.length; i++) {
    let complement = target - nums[i];
    if (complement in numToIndex) {
      return [numToIndex[complement], i];
    }
    numToIndex[nums[i]] = i;
  }
}

// Example usage:
let nums = [2, 7, 11, 15];
let target = 9;
console.log(twoSum(nums, target));  // Output: [0, 1]
```

4) Difference in Population Year Wise

```
const uspop={
  "data": [
    {
      "ID Nation": "01000US",
      "Nation": "United States",
      "ID Year": 2019,
```

```
      "Year": "2019",
      "Population": 328239523,
      "Slug Nation": "united-states"
    },
    {
      "ID Nation": "01000US",
      "Nation": "United States",
      "ID Year": 2021,
      "Year": "2021",
      "Population": 325719178,
      "Slug Nation": "united-states"
    },
    {
      "ID Nation": "01000US",
      "Nation": "United States",
      "ID Year": 2022,
      "Year": "2022",
      "Population": 3257459153,
      "Slug Nation": "united-states"
    },
    {
      "ID Nation": "01000US",
      "Nation": "United States",
      "ID Year": 2020,
      "Year": "2020",
      "Population": 325719178,
      "Slug Nation": "united-states"
    }
  ]
}


let pop_chg_arr = [], pop_arr = {};
const uspopArray = uspop.data.sort((a, b) => a["Year"] - b["Year"]);
uspopArray.forEach((val, index) => {
    let year = parseInt(val['Year']);
    let prevYear = year - 1;
    let diff = null;

    // Check if the previous year exists in the pop_chg_arr
    if (pop_arr[prevYear]) {
        diff = val['Population'] - pop_arr[prevYear];
    }
    pop_arr[year] =val['Population'];
    pop_chg_arr.push({
        "Year": year,
        "Population Difference": diff
    });
});
```

```
console.log(pop_chg_arr);
console.log(pop_arr);
```

5)

To solve the problem of sorting an array in decreasing order based on the count of set bits (1's) in their binary representation, we can approach it in the following steps:

Steps:

1. **Count the number of set bits** for each number in the array.
2. **Sort the array** based on this count, keeping in mind that numbers with the same number of set bits should maintain their original order (this can be achieved using a stable sorting algorithm).
3. **Output the sorted array** after sorting by the number of set bits.

Key Details:

- You can count the number of set bits in a number using **`toString(2)`** to convert the number to its binary representation or use bit manipulation techniques.
- Sorting should be done in decreasing order of set bits.
- If two numbers have the same number of set bits, their original order in the array should be preserved (this is known as **stable sorting**).

Algorithm:

1. For each element in the array, compute the number of set bits.
2. Use a sorting algorithm that sorts primarily by the number of set bits (in decreasing order) and secondarily by the original position if there's a tie.

Solution in JavaScript:

Copy code

```
// Function to count the number of set bits in the binary representation
function countSetBits(num) {
    let count = 0;
    while (num > 0) {
        count += num & 1; // Check if the last bit is 1
```

```
        num = num >> 1;    // Right shift the number
    }
    return count;
}

// Function to sort the array based on the count of set bits
function sortBySetBits(arr) {
    // Sort the array with a custom comparator
    return arr.sort((a, b) => {
        const bitsA = countSetBits(a);
        const bitsB = countSetBits(b);

        // First, compare based on set bits in descending order
        if (bitsA !== bitsB) {
            return bitsB - bitsA; // Sort by set bits in descending order
        }

        // If set bits are equal, preserve the original order (stable sor
        return 0;
    });
}

// Example usage
let arr1 = [2, 4, 8];
let arr2 = [4, 3, 8];

console.log(sortBySetBits(arr1));  // Output: [2, 4, 8]
console.log(sortBySetBits(arr2));  // Output: [3, 4, 8]
```

Explanation:

1. **countSetBits function**: This function uses bitwise operations to count the number of set bits in the binary representation of a number. The `num & 1` checks if the last bit is `1`, and then we right-shift the number using `num >> 1` to process the next bit.
2. **sortBySetBits function**:

- We use JavaScript's built-in `.sort()` function, which allows a custom comparator function.
- Inside the comparator, we first compare the numbers based on the count of set bits. If the counts are different, we return the difference to sort them in descending order.

- If the counts are the same, we return `0` to maintain the original order (stable sorting).

Time Complexity:

- **Counting set bits**: For each number, counting the set bits takes O(log(num)), which is the number of bits in the number. Since `arr[i]` can be as large as 10^9, the number of bits is at most 30 (because $2^{30} > 10^9$).
- **Sorting**: Sorting takes O(N log N), where N is the length of the array. Each comparison in sorting takes O(1) time because the number of bits in any number is at most 30.

  Thus, the overall time complexity is $O(N \log N)$, where N is the number of elements in the array.

Space Complexity:

- O(N) to store the original array and the set bit counts for each number during sorting.

6)

To find all root-to-leaf paths in a binary tree, you can use a **Depth-First Search (DFS)** approach. A root-to-leaf path is any path that starts from the root of the tree and ends at a leaf node (a node with no children).

Problem:

Given a binary tree, return all root-to-leaf paths as an array of strings, where each string represents a path.

Approach:

1. Use DFS to explore the tree.
2. As you traverse each node, keep track of the current path.
3. When you reach a leaf node, add the current path to the list of results.
4. Recursively explore left and right children.

Example:

Given the binary tree:

```
   1
  / \
 2   3
  \
   5
```

The root-to-leaf paths are:

```
["1->2->5", "1->3"]
```

JavaScript Code Implementation:

```javascript
function binaryTreePaths(root) {
  let paths = [];

  function dfs(node, path) {
    if (!node) return;

    // Add the current node value to the path
    path += node.val;

    // If it's a leaf node, add the path to the result
    if (!node.left && !node.right) {
      paths.push(path);
    } else {
      // Otherwise, continue to explore the left and right subtrees
      path += "->"; // Add the arrow symbol for further paths
      dfs(node.left, path);
      dfs(node.right, path);
    }
  }

  dfs(root, ""); // Start DFS from the root with an empty path
  return paths;
}

// Example usage:
class TreeNode {
  constructor(val, left = null, right = null) {
```

```
    this.val = val;
    this.left = left;
    this.right = right;
  }
}

// Creating the example binary tree:
//     1
//    / \
//   2   3
//    \
//     5
let root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(3);
root.left.right = new TreeNode(5);

console.log(binaryTreePaths(root)); // Output: ["1->2->5", "1->3"]
```

Explanation:

1. **DFS Recursive Function (`dfs`)**:

- If the current node is `null`, simply return.
- Add the current node's value to the current path string.
- If the current node is a leaf node (both left and right children are `null`), add the complete path to the list of paths.
- Otherwise, append `"->"` to the path and recursively call `dfs` on the left and right children.

2. **Base Case**:

- If the node is a leaf node, the current path is completed, and it's added to the result.

3. **Helper Function**:

- We use an internal helper function to perform DFS, which keeps track of the path so far and appends the node values to it.

Time and Space Complexity:

- **Time Complexity**: O(n), where `n` is the number of nodes in the binary tree. We visit each node once.

- **Space Complexity**: O(h), where `h` is the height of the tree due to the recursion stack. In the worst case (skewed tree), it could be O(n), but in a balanced tree, it would be O(log n). The space complexity for storing paths is also O(n).

7) BellmanFord

```
function bellmanFord(graph, source) {
    // Step 1: Initialize distances from source to all vertices as
infinite and distance to the source itself as 0
    let distance = {};
    for (let vertex in graph) {
        distance[vertex] = Infinity;
    }
    distance[source] = 0;

    // Step 2: Relax all edges |V| - 1 times. A simple shortest path
from the source to any other vertex can have at most |V| - 1 edges
    for (let i = 1; i <= Object.keys(graph).length - 1; i++) {
        for (let u in graph) {
            for (let [v, weight] of graph[u]) {
                if (distance[u] + weight < distance[v]) {
                    distance[v] = distance[u] + weight;
                }
            }
        }
    }

    // Step 3: Check for negative-weight cycles. The above step
guarantees the shortest distances if the graph doesn't contain a
negative weight cycle.
    for (let u in graph) {
        for (let [v, weight] of graph[u]) {
            if (distance[u] + weight < distance[v]) {
                console.log("Graph contains a negative-weight cycle");
                return;
            }
        }
    }

    return distance;
}


let graph = {
    A: [['B', 4], ['C', 2]],
    B: [['C', 5], ['D', 10]],
    C: [['E', 3]],
```

```
    D: [['F', 11]],
    E: [['D', 4]],
    F: []
};

let source = 'A';
let distances = bellmanFord(graph, source);
console.log(distances);
```

8) let allowed = "ab"; let words = ["ad","bd","aaab","baa","badab"]; change code using reduce

To refactor your code to use `reduce` properly and count the consistent strings, you can simplify the approach by adjusting how the accumulator works. Here's an updated version of your code:

Copy code

```
var countConsistentStrings = function(allowed, words) {
    const allowedSet = new Set(allowed); // Use a Set for faster lookups
    return words.reduce((tot_count, word) => {
        const isConsistent = word.split('').every(char => allowedSet.has(
        return isConsistent ? tot_count + 1 : tot_count;
    }, 0);
};

let allowed = "ab";
let words = ["ad","bd","aaab","baa","badab"];
console.log(countConsistentStrings(allowed, words)); // Output: 2
```

Explanation:

- `allowedSet`: A `Set` is used to hold the allowed characters since lookups in a `Set` are faster.
- `reduce`: It processes the array `words`, checking each word to see if all its characters are in `allowedSet`.
- `every`: This method ensures all characters of the word are valid (i.e., present in `allowedSet`). If true, the counter is incremented; otherwise, it remains the same.

This approach efficiently counts the consistent strings with fewer checks and faster performance.